



Strings & Text

Python Mastery — Part 1: Foundations · Week 3

Rathinam Trainers & Consultants

Live on Microsoft Teams · recorded

Recap

Where we left off last week

Last week — Numbers

- ▶ Numeric types: `int`, `float`, `complex`, `bool`
- ▶ `/` vs `//` vs `%`, `**`, and **operator precedence**
- ▶ Variables **bind names to objects**; `None` = absence of a value
- ▶ Floats are **approximate** — `round()`, formatting, `math.isclose`

You can already use Python as a calculator. Today we make it talk.

Today's goal

By the end of this session you'll understand how to:

- ▶ **Index and slice** strings; use the everyday **methods**
- ▶ Explain why strings are **immutable**
- ▶ Build clean output with **f-strings** + the **format mini-language**
- ▶ Recognise a **t-string** and how it differs from an f-string
- ▶ Say what **bytes** vs **str** means and why **encodings** exist

Strings, up close

Literals, indexing, slicing, methods

What a string is

- ▶ A `str` is an **ordered sequence of characters** — text
- ▶ Make one with quotes: `'single'` or `"double"` (same thing)
- ▶ **Triple quotes** for multi-line: `"""..."""`
- ▶ **Escapes:** `\n` newline, `\t` tab, `\\` backslash
- ▶ **Raw strings:** `r"C:\new"` — backslashes stay literal

Indexing — reach one character

```
>>> word = "Python"  
>>> word[0]  
'P'  
>>> word[-1]  
'n'
```

- ▶ Counting starts at **0**, not 1
- ▶ **Negative** indexes count from the right: **-1** is the last
- ▶ Out-of-range index → `IndexError`

Slicing — reach a substring

```
>>> word[1:4]
'yth'
>>> word[:2]
'Py'
>>> word[::-1]
'nohtyP'
```

- ▶ `s[start:stop]` — `stop` is **excluded**
- ▶ Omit either side for "from the start" / "to the end"
- ▶ Third number is the **step**; `[::-1]` reverses

High-value string methods

Method	Does
<code>.strip()</code>	trim surrounding whitespace
<code>.lower()</code> / <code>.upper()</code> / <code>.title()</code>	change case
<code>.split(sep)</code> / <code>sep.join(list)</code>	text ↔ list
<code>.replace(a, b)</code>	swap substrings
<code>.find()</code> / <code>.index()</code>	locate text
<code>.startswith()</code> / <code>.endswith()</code>	test edges

Strings are immutable

```
>>> word[0] = "J"  
TypeError: 'str' object does not  
support item assignment
```

- ▶ A string **cannot be changed in place**
- ▶ Methods **return a new string**; the original is untouched
- ▶ This is *why* we chain: each step hands a fresh string on

Let's clean up some messy text.

Watch this — Demo 5

Slicing & method chaining

- ▶ Take `" Buy Milk, Eggs, Bread "`
- ▶ `.strip()` → `.lower()` → `.split(",")`
- ▶ Index & slice a word: `[0]`, `[-1]`, `[:3]`, `[::-1]`
- ▶ Rebuild clean with `", ".join(...)`
- ▶ Prove the **original is unchanged**

Method chaining, step by step

```
>>> raw = "  Buy Milk, Eggs, Bread  "
>>> raw.strip().lower()
'buy milk, eggs, bread'
>>> items = [w.strip() for w in
...           raw.strip().split(",")]
>>> items
['Buy Milk', 'Eggs', 'Bread']
>>> ", ".join(items)
'Buy Milk, Eggs, Bread'
```

Each method returns a **new** string — `raw` never changes.

What you just saw

- ▶ One messy string, **reshaped by chaining** methods
- ▶ `split` turns text into a list; `join` turns a list into text
- ▶ Indexing and slicing pull out exactly the piece you want
- ▶ The original string stayed put — methods **return new strings**

Formatting output

f-strings & the format mini-language

f-strings — the everyday tool

```
>>> name = "Eggs"  
>>> qty = 12  
>>> f"{name}: {qty}"  
'Eggs: 12'
```

- ▶ Prefix a string with `f`, put **expressions** in `{ }`
- ▶ Any expression works: `f"{qty * 2}"`, `f"{name.upper()}"`
- ▶ It evaluates **immediately** to an ordinary `str`

Self-documenting {value=}

```
>>> qty = 12
>>> f"{qty=}"
'qty=12'
>>> f"{qty * 2 =}"
'qty * 2 =24'
```

- ▶ Add `=` inside the braces → prints **the expression and its value**
- ▶ A brilliant, fast **debugging** trick — no more `print("qty", qty)`

The format mini-language

```
{ value : [align][width][,][.prec][type] }
```

- ▶ **Width + align:** {name:<10} left, {n:>8} right, {x:^6} centre
- ▶ **Thousands grouping:** {1234567:,} → 1,234,567
- ▶ **Fixed decimals:** {3.14159:.2f} → 3.14
- ▶ **Combine:** {price:>10,.2f}

Let's format a tidy report line.

Watch this — Demo 6

f-string formatting + t-string contrast

- ▶ Build an aligned row: `f"{name:<10}{qty:>4}{price:>8.2f}"`
- ▶ Show `,` grouping and `{value=}` debugging
- ▶ Write the same text as a `t"..."` t-string
- ▶ `type()` it → it's a `Template`, not a `str`
- ▶ Say in one line **what t-strings are for**

Aligned report row

```
>>> name, qty, price = "Eggs", 12, 3.5
>>> f"{name:<10}{qty:>4}{price:>8.2f}"
'Eggs      12    3.50'
>>> f"{1234567.5:>12,.2f}"
'1,234,567.50'
```

- ▶ `<10` pads left to width 10; `>8.2f` right-aligns, 2 decimals
- ▶ Columns line up because every field has a **fixed width**

f-string vs t-string

```
>>> name = "Eggs"
>>> f"Item: {name}"          # f-string
'Item: Eggs'
>>> t"Item: {name}"          # t-string
Template(strings=('Item: ', ''),
          interpolations=(...))
>>> type(t"Item: {name}")
<class 'string.templatelib.Template'>
```

- ▶ f-string → an immediate **str**
- ▶ t-string → a **Template** you can inspect first

Why t-strings exist (one beat)

- ▶ A `Template` keeps the **static parts** and the **values** separate
- ▶ That lets library code **sanitise values before** building text
- ▶ The classic win: stop **SQL injection / HTML (XSS)** bugs
- ▶ New in **3.14** (PEP 750). In Part 1 we just **recognise** them

What you just saw

- ▶ One f-string produced an aligned, grouped, 2-decimal row
- ▶ `{value=}` printed both the expression and its value
- ▶ The **same** interpolation as a `t"..."` made a `Template`, not a `str`
- ▶ You can now state how a t-string differs from an f-string

Bytes & encodings

Why text needs a translation step

str vs bytes

- ▶ A `str` is **human text** — characters
- ▶ `bytes` is **raw computer storage** — numbers 0–255
- ▶ Files and networks move **bytes**, not characters
- ▶ So we need a rule to convert between them: an **encoding**

Encode / decode with UTF-8

```
>>> s = "café"
>>> data = s.encode("utf-8")
>>> data
b'caf\xc3\xa9'
>>> data.decode("utf-8")
'café'
```

- ▶ **encode:** `str` → `bytes` (text out to storage)
- ▶ **decode:** `bytes` → `str` (storage back to text)
- ▶ **UTF-8** is the standard default — use it unless told otherwise

What you just saw

- ▶ Text and stored bytes are **two different things**
- ▶ An **encoding** is the agreed rule that maps between them
- ▶ `.encode()` goes text → bytes, `.decode()` goes bytes → text
- ▶ This matters the moment we touch **files** in Week 12

Wrap-up

This week's homework

Lab 3 — a text-report formatter

1. Write `report.py` with a few `(name, quantity, price)` rows
2. Clean any messy fields with `.strip()` / slicing (Demo 5)
3. Print an **aligned table** with f-strings + the mini-language
4. Numbers show **2 decimals** and **thousands grouping**
5. Add a short comment sketching **one line as a t-string** + why type differs

Mirror **Demo 5** then **Demo 6** from the recording.

Next week

- ▶ **Week 4 — Control Flow & Pattern Matching**
- ▶ We'll use these strings to **build conditions and loop bodies**
- ▶ `match / case` will parse command strings like the ones you cleaned today

See you in the lab.

Questions?